

2022 年面向对象程序设计期末考试参考答案

一、选择题

1.

答案: C

逐项看:

- A 中 `const std::string message;` 合法, 表示默认构造出一个空字符串
- B 中 `const std::string hello("Hello");` 合法
- C 中 `std::string` 的这个构造函数第二个参数应是单个字符, 例如 `std::string exclam(5, '!')`, 不能写成字符串字面量 `"!"`
- D 中迭代器区间构造也是合法的, 得到的是 `"orl"`

因此错误的是 C。这一题主要考察 `string` 的几种常见构造方式。

2.

答案: B

四行代码分别判断如下:

1. `const int buf;` 错, 常量必须初始化
2. `int cnt = 0;` 对
3. `const int sz = cnt;` 对
4. `++cnt; ++sz;` 这一行里前半句 `++cnt` 本身可以成立, 但 `++sz` 不行, 因为 `sz` 是常量

因此正确的只有 2 行。

3.

答案: B

三组函数中:

- (a) 可以重载, 因为参数类型分别是 `int&` 和 `const int&`
- (b) 不可以重载, 因为只有返回类型不同, 参数列表完全相同
- (c) 可以重载, 因为参数类型分别是 `int*` 和 `double*`

所以能正确重载的是 a 和 c。

4.

答案: A

这段程序在编译阶段就过不去, 原因至少有两处:

1. 类 A 只有带一个 `int` 参数的构造函数, 没有默认构造函数, 但 `A a[10];` 需要默认构造 10 个对象
2. `Return 0;` 中的 `Return` 大小写错误, C++ 关键字应写成 `return`

因此应选编译错误。

5.

答案：D

vector 和 list 都是顺序容器，都能动态增长，也都支持 begin()、end() 和 push_back()。
但标准库算法 sort() 需要随机访问迭代器，list 不满足这个条件，因此不能写成 sort(lst.begin(), lst.end())
这种形式。list 要排序，应使用它自己的成员函数 list::sort()。

6.

答案：B

如果 cond 成立，那么执行到 throw exception 后就会立刻离开 try 块，block A 不会执行，程序转而进入匹配的 catch 块执行 block B。

如果 cond 不成立，就正常执行 block A，不会进入 catch。

不管有没有异常，后面的 block C 最终都会继续执行。

因此错误的是 B。

7.

答案：D

正确的是第 2 条和第 4 条：

- 第 2 条：虚函数通常会带来额外空间和时间开销
- 第 4 条：虚函数依赖动态绑定

第 3 条错在“静态绑定”；虚函数恰恰是为运行时多态服务的。

第 1 条说法太笼统。虚函数确实通常会比普通非虚调用多一些开销，但不能简单理解成“运行开销大”。

8.

答案：C

delete 删除动态对象时会自动调用析构函数；删除动态数组时应写成 delete[] p。

但对同一个指针重复 delete 是非常危险的，属于未定义行为，可能导致程序崩溃。因此 C 错。

二、程序阅读题

2.1

输出结果：

13,9,11

这里考察的是引用参数、指针参数和值传递参数的区别。

初值为：

$a = 5, b = 7, c = 11$

调用 $f(a, \&b, c)$ 后:

1. x 绑定到 a , y 指向 b , z 只是 c 的一个副本
2. $z = x + *y / 2 = 5 + 7 / 2 = 8$
3. $*y = x + z / 2 = 5 + 8 / 2 = 9$, 所以 $b = 9$
4. $x = *y + z / 2 = 9 + 8 / 2 = 13$, 所以 $a = 13$
5. c 没有被改动, 因为它是按值传入的

所以最后输出 13,9,11。

2.2

输出结果:

ABD

ABD

更准确地说, 是两行完全相同的输出, 每行末尾都有换行。

分析如下:

1. `new Derived[2]` 创建两个 `Derived` 对象
2. 默认参数 $n = 'A'$
3. 派生类构造时调用 `Base(n + 1)`, 因此基类中
 - $c1 = 'B'$
 - $c2 = 'D'$
4. 派生类自己的 $c3 = 'A'$

删除数组时按**逆序**析构, 每个对象都先执行派生类析构函数, 再执行基类析构函数, 所以每个对象都会输出:

A

BD

连起来正好是:

ABD

由于一共有两个对象, 因此总输出就是两行 ABD。

2.3

输出结果:

5

51

这题主要考察静态成员、复制构造和赋值运算符。

先看 `seq` 的变化。静态数据成员 `seq` 初值为 0:

1. Abc m 后, seq = 1, m.value_ = 1
2. Abc n 后, seq = 2, n.value_ = 2
3. Abc p 后, seq = 3, p.value_ = 3
4. Abc q(p) 调用复制构造, q.value_ = p.value_ + 0x64 = 3 + 100 = 103, 同时 seq = 4
5. m = q 调用赋值运算符, 先 seq++ 变成 5, 再令 m.value_ = 103 / 2 = 51

因此:

- p.seq 访问的是类的静态成员, 输出 5
- m.value() 输出 51

2.4

输出结果:

```
6 4 2 0 5 3 1
0 0 0 0 5
```

这题比较有代表性, 考察了构造、copy_if 和 back_inserter 的共同作用。

数组 a 中各元素经过构造函数 Value(int v = 0) : value_(v % 7) 后, values 中实际保存的是:

```
6 4 2 0 5 3 1
```

filter 在 copy_if 过程中会对每个元素先输出它的值, 再判断是否满足 value % 5 == 0。因此第一行一定是:

```
6 4 2 0 5 3 1
```

接着注意这一句:

```
vector<Value> filtered(values.size() / 2);
```

这不是“预留容量”, 而是直接创建了 3 个默认元素, 也就是先放入了三个值为 0 的 Value 对象。

之后 copy_if(..., back_inserter(filtered), filter) 又把满足条件的元素追加到末尾。满足条件的只有 0 和 5, 于是 filtered 最终内容变成:

```
0 0 0 0 5
```

所以第二行输出正是这五个数。

三、程序填空题

填空答案如下:

- (1) `template <class T>`
- (2) `: data(capacity + 1)`
- (3) `last >= first`
- (4) `e = data[first]`
- (5) `first = (first + 1) % data.size()`
- (6) `data[last] = e`

```
(7) last = (last + 1) % data.size()
(8) cout << "<" << data[i] << ">"
```

说明如下：

1. 若最大可容纳元素个数为 N ，就要实际分配 $N + 1$ 个存储单元，用额外空出的一个位置来区分“空”和“满”
2. `getLength()` 中，如果 `last >= first`，长度直接是 `last - first`
3. 出队时应先取出队头元素，再把 `first` 向后移动一格
4. 入队时应先把新元素写入 `last` 所指位置，再移动 `last`
5. `print()` 按循环队列当前顺序依次输出即可

按题目主程序来验证：

1. `LoopQueue<int> qu(3)` 最多容纳 3 个元素
2. 循环入队 1 到 5 时，只有 1、2、3 真正成功
3. 出队后取出 1，队列中剩 2、3
4. 再把 1 入队，得到 2、3、1

因此输出确实为：

```
2
3
<2><3><1>
```

四、编程题

1. 模板函数 `Rotate(begin, mid, end)`

这一题主要考察模板函数、迭代器和区间操作。题面特别说明 `begin`、`mid`、`end` 为**前向迭代器**，所以解法最好不要依赖随机访问能力，例如 `end - begin` 这样的写法并不稳妥。

一种朴素但完全可行的办法，是先把区间 `[begin, mid)` 的元素暂存起来，再把 `[mid, end)` 的元素往前复制，最后把暂存部分接到末尾。参考实现如下：

```
#include <vector>
#include <iterator>

template <class ForwardIter>
void Rotate(ForwardIter begin, ForwardIter mid, ForwardIter end)
{
    typedef typename std::iterator_traits<ForwardIter>::value_type value_type;

    std::vector<value_type> temp;
    ForwardIter it;

    for (it = begin; it != mid; ++it)
        temp.push_back(*it);
```

```

ForwardIter dest = begin;
for (it = mid; it != end; ++it, ++dest)
    *dest = *it;

for (typename std::vector<value_type>::size_type i = 0; i != temp.size(); ++i, ++dest)
    *dest = temp[i];
}

```

若原序列是：

1 2 3 4 5 6 7 8 9

且 mid 指向 4，那么旋转后就变成：

4 5 6 7 8 9 1 2 3

与题目给出的输出一致。

2. 夏令营选拔

这一题综合性较强，主要考察结构体、类封装、排序规则、读入输出，以及“按分数线截取一段有序数据”的思路。题目已经规定了排序规则：

1. 先按笔试成绩从高到低
2. 成绩相同按报名号从小到大

因此比较函数必须严格按这两层规则写。

另外，面试分数线不是简单取前 m 个，而是先计算 $k = \text{floor}(1.5 * m)$ ，再取第 k 名的分数作为分数线，最后把所有不低于这个分数线的人都算入围。

下面给出一份符合题意的参考实现。

candidates.h

```

#ifndef CANDIDATES_H
#define CANDIDATES_H

#include <iostream>
#include <vector>
#include <string>

struct candidate {
    int id;
    int score;

    candidate(int i = 0, int s = 0) : id(i), score(s) {}
};

```

```

class candidates {
public:
    candidates();

    void readinfo(std::istream& in);
    void sort_by_score_ID();
    void calculate(int m);
    void output(std::ostream& os) const;

    int get_mps() const;
    int get_num() const;

private:
    std::vector<candidate> data;
    int minimum_passing_score;
    int interview_num;
};

```

```

#endif

```

candidates.cpp

```

#include "candidates.h"
#include <algorithm>

```

```

using std::istream;
using std::ostream;

```

```

candidates::candidates() : minimum_passing_score(0), interview_num(0) {}

```

```

void candidates::readinfo(istream& in)
{
    int id, score;
    while (in >> id >> score) {
        data.push_back(candidate(id, score));
    }
    in.clear();
}

```

```

static bool cmp_candidate(const candidate& a, const candidate& b)
{
    if (a.score != b.score)

```

```

        return a.score > b.score;
    return a.id < b.id;
}

void candidates::sort_by_score_ID()
{
    std::sort(data.begin(), data.end(), cmp_candidate);
}

void candidates::calculate(int m)
{
    int k = m * 3 / 2;
    minimum_passing_score = data[k - 1].score;

    interview_num = 0;
    for (std::size_t i = 0; i != data.size(); ++i) {
        if (data[i].score >= minimum_passing_score)
            ++interview_num;
    }
}

void candidates::output(ostream& os) const
{
    for (int i = 0; i != interview_num; ++i) {
        os << data[i].id << " " << data[i].score << std::endl;
    }
}

int candidates::get_mps() const
{
    return minimum_passing_score;
}

int candidates::get_num() const
{
    return interview_num;
}

```

按照上述方法设计程序，输出结果就与题面一致。